Système d'exploitation : TP6
Entrées-sorties bas niveau

mars 2026 — Pierre Rousselin

Dans les exercices de ce sujet, on utilisera les appels système `open`, `close`, `read` et `write`. Pour des raisons pédagogiques, on évitera le plus possible d'utiliser les fonctions de plus haut niveau `printf`, `scanf`, ...

Exercice 1 : Ouvrir un fichier et y lire des octets

1. Dans le répertoire courant, écrire dans un fichier nommé `lol.txt` les deux lignes
   ```
   hop !
   plop !
   ```
   (l'espace est une espace normale et un point d'exclamation termine chaque ligne).
2. Quelle est la taille en octets de ce fichier si les fins de ligne sont de la forme LF (*Line Feed*, c'est-à-dire un seul caractère `'\n'`), et si les fins de ligne sont de la forme CRLF (*Carriage Return, Line Feed*, c'est-à-dire les caractères `'\r'` puis `'\n'`) ?
3. Télécharger, lire, compiler et exécuter le programme `open_read.c` présent sur *moodle*.
4. Ajouter les instructions permettant de lire d'un coup 4 octets et de les écrire dans le terminal (attention à ajouter ce qu'il faut dans `buffer` pour que `printf("%s")` puisse fonctionner). Quelle est la valeur de retour de `read` ?
5. Demander à `read` de lire les 20 octets suivants et de les mettre dans `buffer`. Afficher la valeur de retour de `read`, puis le contenu pertinent de `buffer`.
6. Demander à `read` de lire encore 1 octet dans le fichier. Quelle est la valeur de retour de `read` ?
7. L'appel système `write` permet d'écrire dans un fichier. Son prototype est le suivant :
   ```
   ssize_t write(int fd, const void *buf, size_t count);
   ```
   Précisément, cet appel système écrit les `count` premiers octets à partir de l'adresse `buf` dans le fichier de descripteur `fd`. Le descripteur de fichier associé à la sortie standard (*a priori*, écriture dans le terminal) est 1.

   Modifier votre programme de manière à ce que les appels à `printf` qui affichent le contenu du fichier `lol.txt` soient remplacés par des appels système `write`.

--- * ---

Exercice 2 : `mon_cat`

Écrire votre propre version de `cat` avec `read` et `write` : lire les fichiers dont les noms sont donnés en arguments et les écrire à la suite (con**cat**éner) sur la sortie standard.

Faire les tests d'usage pour les ouvertures de fichiers (avec messages d'erreur utiles) et comparer avec ce que fait le `cat` installé sur votre machine.

Sans argument, votre processus doit lire l'entrée standard.

--- * ---

Exercice 3 : Ouvrir un fichier en écriture

1. Télécharger sur moodle, lire, compiler et exécuter le fichier `create.c`
2. Afficher le contenu des fichiers `lol1` et `lol2` ainsi que leurs permissions. Pour comprendre les permissions associées à `lol1`, penser au masque utilisateur (commande `umask`).
3. Exécuter de nouveau le programme `create.c` et expliquer les différences entre les contenus de `lol1` et de `lol2`.

--- * ---

Exercice 4 : `mon_cp`

Écrire avec les appels systèmes `read` et `write` votre propre version (minimale) de `cp`. Votre `cp` simplifié aura toujours deux arguments : le nom du fichier source et le nom du fichier destination, et il recopiera si possible le contenu du fichier source dans le fichier destination.

`--- * ---`